ChainProv-City: An Authority-Based Blockchain Provenance Framework for Trustworthy Event Verification in Smart Cities

Maruf Farhan[1*], Madhuki Rajapakshe[2], Usman Butt[3] and Rejwan Bin Sulaiman[4]

[1] Ulster University, London, UK
[2] Coventry University, Dagenham, UK
[3] College of Engineering and Information Technology, Ajman University, UAE
[4] Northumbria University, London, UK
m.farhan@ulster.ac.uk

Abstract. Data-driven smart cities increasingly rely on large-scale data streams processed by AI for decision-making, but current systems often rely on centralised data structures susceptible to tampering, lack of auditability, and new threats like deepfakes. In this paper, we introduce ChainProv-City, an authority-focused blockchain provenance model to enhance trust and accountability in smart city surveillance systems. The new approach, rather than relying on anonymous consensus, incorporates institutional authorities into the verification process, so that verification reflects real-world governance. The system combines a formal model, algorithmic design of a lifecycle, and a multi-agency verification process with threshold consensus. Security is evaluated against realistic adversaries and a scenario-based case study shows resilience to misinformation attacks. An Ethereum-based prototype proves the approach is feasible with minimal infrastructure. In summary, ChainProv-City offers a governance-sensitive framework for verifiable event management, linking blockchain provenance to smart cities.

Keywords: Blockchain, Smart Cities, Data Provenance, Multi-Agency Verification, AI Surveillance, Deepfake Detection, Governance-Aware Systems.

1 Introduction

Smart cities generate vast amounts of digital event data from diverse sources (e.g., IoT devices, surveillance, transport, analytics). This data supports critical decisions such as emergency management, traffic control and public safety. So, integrity, auditability and accountability are essential [1, 2]. However, most systems are still centralised, often based on municipal databases or cloud platforms. They are efficient but can be susceptible to insider fraud, data manipulation, and auditability issues, especially in surveillance applications [3]. These problems are compounded by AI-driven disinformation (e.g., deepfakes, synthetic data) that exploits automation bias in decision-making [7-9].

Blockchain could help address these issues through its append-only, immutable nature and distributed trust. Existing research has considered blockchain-based

provenance in IoT, digital forensics, and smart cities [6-8]. Most approaches adopt open or consensus-based models, where verification is done by unknown parties or incentives. These approaches work for permissionless systems, but not for urban governance, where trust is institutional and legally accountable [9]. In reality, smart city systems are governed by institutions. Government agencies like police, transport and emergency services have established charters. Modelling these as anonymous users is not representative of the real world. Lack of explicit authority modelling limits use [10, 11].

To overcome this, this work presents ChainProv-City, an authority-based blockchain provenance model for smart cities. Rather than open validation, verification is limited to authorities and decisions are stored on the blockchain. This approach uses blockchain as a coordination and accountability layer, not a replacement for institutional trust.

Our contributions are (i) an authority-based provenance model compatible with governance, (ii) algorithms and mathematical proofs for correct and irreversible state transitions, (iii) a threat model for adversarial risks, (iv) a scenario-based case study, and (v) a lightweight implementation on Ethereum. To the beset of our knowledge, ChainProv-City is the first to explicitly represent institutional authority in the chain of provenance of smart city surveillance systems. The rest of the paper is organised as follows: Section 2 reviews related work; Section 3 describes the architecture; Section 4 the algorithms and lifecycle; Section 5 the security model; Section 6 implementation and validation; Section 7 a case study and discussion; and Section 8 concludes the paper.

2 Background and Related Work

2.1 AI-Driven Smart Cities and Core Challenges

Cognitive cities build on the smart city concept by incorporating artificial intelligence and machine learning technologies to build systems that can learn, adapt and predict [5], [12]. These systems facilitate proactive and autonomous decision-making, whereas traditional rule-based systems do not, as in the case of traffic management and surveillance systems, which use real-time and historical data to inform actions [12]. Reliability is essential as they affect public safety and urban operations. But several challenges remain. Data authenticity is still critical, since AI-based decisions depend on reliable data, but urban sensing is inherently noisy and prone to faults and attacks, such as deepfakes. Provenance and auditability is also critical for accountability, but many current systems rely on centralised or unstandardised logging, which can be vulnerable to tampering or loss [10], [11]. However, transparency also needs to be tempered by privacy, as logging may reveal sensitive or identifying metadata [8]. Other threats, such as denial-of-service, malware, and insider abuse, also require fault-tolerant verification [12].

2.2 Blockchain for Provenance and Verification

Blockchain has been widely studied to enhance trust, integrity and provenance in smart cities. The append-only nature, cryptographic security and decentralisation of blockchain allow for tamper-proof audit trails and provenance [3], [10]. Previous work has used blockchain to support IoT-based service sharing [1] and authentication and authorisation in heterogeneous smart cities [2]. Meanwhile, blockchain-based provenance

has been extensively explored in digital forensics. Solutions like ForensiBlock [4] show how permissioned blockchains can be used to provide traceability and access control through role-based access control. But these systems are generally post-incident, while smart city surveillance needs to be processed frequently and verified close to real time.

Furthermore, multi-party verification reduces single points of failure and enhance trust. Blockchain governance extends this to collaborative IoT access control [5]. However, current solutions focus on access policy enforcement rather than ensuring event correctness before operational decisions. This is critical in surveillance, where incorrect events can trigger harmful actions, and the lack of explicit authority modelling limits use in government systems [6]. Recent research has addressed AI-driven misinformation using blockchain-based media verification. Methods incorporating watermarking, digital signatures and federated learning aim to authenticate media [7-9], but are not designed for the scale, speed and operational challenges of urban surveillance.

2.3 Limitations of Existing Approaches and Research Gap

While there is much research in this area, it only addresses some aspects. Forensic blockchain models provide integrity and traceability, but lack real-time, multi-agency support [4], and smart city blockchain systems enhance scalability but rely on a single verifier [1],[2]. Deepfake detection improves content verification but lacks scalability [8]. Lacking a single framework for real-time, multi-agency, complete provenance, privacy-preserving and adversary-resistant operation, we propose ChainProv-City.

Table 1. Research Gap Analysis: Core Requirements versus Existing Work Categories.

Requirement Category	Real-Time Operation	Multi-Agency Governance	Provenance Completeness	Privacy Compliance	Resilience to Adversarial Manipulation
Smart city blockchain [1], [2]	Yes	No	Partial	No	No
Forensic blockchain systems [4]	No	No	Yes	Partial	Partial
Multi-party access control [5]	Partial	Yes	No	Partial	No
Deepfake detection/blockchain [8]	No	No	Yes	No	Yes
Gap addressed by ChainProv-City	Yes	Yes	Yes	Yes	Yes

3 System Architecture and Workflow

This section describes ChainProv-City architecture and interactions. ChainProv-City has 3 layers: (i) data acquisition and edge processing, (ii) blockchain-based provenance, and (iii) human and institutional oversight. They facilitate a workflow for event recording, provenance capture and multi-party verification.

3.1 System Model and Formal Definitions

The ChainProv-City system is defined as:

$$S = (E, A, V, L, T) \quad (1)$$

Where, E is the set of urban events $E_i = (d_i, t_i, s_i, m_i)$ representing data, timestamp, source identifier, and metadata; A is the set of authorized authorities (verifiers); V:E×A→{0,1} is the verification function; L is a permissioned blockchain ledger with smart contract support; and T is the threshold k ≤ |A| required for event confirmation.

Data are created by diverse sensors, edge processed, with hashes recorded on-chain, and assessed by multiple verifiers. The threshold T enforces multi-party verification, preventing unilateral validation and enhancing accountability and security.

3.2 Layer 1: Data Acquisition and Edge Processing

Events are sourced from CCTV, UAVs or IoT devices which can use on-board AI for event detection or classification, and sent to a trusted edge gateway.

Event Registration Protocol

- Detection: Source device s_i detects event E_i
- Hashing:

$$H(E_i) = \text{SHA-256}(d_i \;||\; t_i \;||\; s_i) \quad (2)$$

- Edge Aggregation: Gateway G_j authenticates sources, performs temporal and spatial deduplication, and aggregates related events into batches

$$B_t = \{E_i\}_{i=1}^{n} \quad (3)$$

- Off-Chain Storage: Raw data stored (e.g., IPFS), producing:

$$\text{CID}(d_i) \quad (4)$$

- Transaction Preparation

$$\tau_i = (H(E_i), CID(d_i), m_i, status = PENDING) \quad (5)$$

Layer 1 acts as a validation boundary, filtering out duplicate information, authenticating sources, and creating the first cryptographic link between data and identity. This lowers costs by eliminating the need to broadcast raw data.

Inter-Layer Communication: Layer 1's validated batches are posted to Layer 2 through smart contract calls, creating receipts R_t that tie off-chain data to the blockchain.

3.3 Layer 2: Blockchain and Provenance Management

Layer 2 is an immutable provenance layer implemented on a permissioned blockchain managed by trusted parties. This is responsible for event registration, authorisation, verification and logging. Three smart contracts are used:

Contract 1: Event Registry. Records hash, content reference, submitter, timestamp, status and verifiers' decisions (Listing 1). Events start with status PENDING. The contract enforces unique registration, one vote per authority and checks are enforced.

```
struct EventRecord {
    bytes32 eventHash;
    string contentCID;
    address submitter;
    uint256 timestamp;
    EventStatus status;
    mapping(address => VerificationVote) votes;
    uint8 confirmationCount;
}
```

Algorithm 1 captures this, ensuring authenticity, uniqueness and verifiable recording in the ledger, while retaining the association with off-chain information.

Algorithm 1: Event Registration in Blockchain Registry

Input : Transaction payload $\tau_i = \langle H(E_i), CID(d_i), m_i, \text{status} \rangle$, Submitter Address $addr_s$
Output: Transaction Receipt R_t or Rejection Error

// Step 1: Ingestion from Edge layer
Receive transaction payload τ_i from the authenticated Edge Gateway;

// Step 2: Security and Validation
if IsAuthorized($addr_s$) = False **then**
  return Error: Unauthorized Submitter;

// Step 3: Integrity Check
if ExistsInRegistry($H(E_i)$) = True **then**
  return Error: Duplicate Event Detected;

// Step 4: Record Creation and State Update
$t_{reg} \leftarrow$ GetCurrentTimestamp();
Create new event record:
$ER_i \leftarrow \{H(E_i), CID(d_i), addr_s, t_{reg}, \text{PENDING}, \text{votes} \leftarrow \emptyset, \text{conf} \leftarrow 0\}$;

// Step 5: On-chain Persistence
StoreOnChain(ER_i);
Emit EventRegistered($eventId, H(E_i), t_{reg}$);
return Transaction Receipt R_t;

Contract 2: Authority Manager. Stores verifier identities and roles via RBAC. It controls which entities can verify events, securing accountability and trust across agencies. Event confirmation follows:

$$\sum_{a \in A} V(E_i, a) \geq k \tag{6}$$

Contract 3: Provenance Tracker. Records all event state transitions and stores signed verification proofs, preserving full lifecycle of events from registration to resolution:

$$\pi_i = \text{Sign}_{sk_a}(H(E_i), decision, t_{verify}) \tag{7}$$

State Machine: The event lifecycle follows a deterministic finite state machine, linking all layers where events originate in Layer 1, are evaluated in Layer 3, and enforced in Layer 2. States and transitions are formally defined as follows:

- PENDING → UNDER_REVIEW
- UNDER_REVIEW → CONFIRMED (if threshold met)
- UNDER_REVIEW → REJECTED (if rejection condition met)

Inter-Layer Communication: State changes are emitted as blockchain events. Layer 3 dashboards subscribe via WebSockets or message queues, enabling real-time updates and reducing manual synchronisation errors between blockchain records and systems.

3.4 Layer 3: Human and Institutional Oversight

This layer introduces governance through human verification and institutional decision-making. Authorities interact via secure dashboards.

Verification Workflow

- Alert: Authority a ∈ A receives event E_i
- Retrieve Data: CID(d_i)
- Integrity Check:

$$H'(E_i) = \text{SHA-256}(d_i \;||\; t_i \;||\; s_i) \tag{8}$$

Verify:

$$H'(E_i) = H(E_i) \tag{9}$$

- Decision:
  - Valid → $V(E_i, a) = 1$ with digital signature σ_a
  - Invalid/manipulated → $V(E_i, a) = 0$ with fraud flag
- Progress monitoring:

$$\frac{votes}{k} \text{ x } 100\% \tag{10}$$

Layer 3 actively participates in the provenance lifecycle, retrieving evidence from Layer 1, verifying it against immutable commitments in Layer 2, and submitting signed decisions to the blockchain for threshold evaluation.

Oversight Functions: Real-time monitoring, audit (post-event) using provenance logs and accountability (signed actions).

Inter-Layer Communication: Interactions are via wallets or enterprise key systems. Write operations invoke smart contracts, while read operations use off-chain indexers (e.g., The Graph) to access event histories and states without incurring full blockchain latency. Events can trigger external city systems.

3.5 Cross-Layer Data Flow and Security Properties

Security is achieved through coordinated interactions across layers:

- Edge → Chain: Hash $H(E_i)$ ensures integrity; tampering is detectable
- Chain → Authority: RBAC + certificates bind identities to actions
- Authority → Edge: Verified events trigger real-world responses

Privacy Design: Sensitive data remains off-chain in encrypted storage. Only:

$$(H(E_i), CID(d_i)) \tag{11}$$

are stored on-chain, preserving integrity verification while minimising exposure of surveillance content, especially data containing personally identifiable information.

Blockchain Agnosticism: The architecture is compatible with platforms such as Hyperledger Fabric, Quorum, or Ethereum-based systems.

3.6 Formal Algorithms

Algorithm 2: Event Verification and State Transition

Input : Event record ER_i, verifier $a \in A$, decision $V(E_i, a) \in \{0, 1\}$
Output: Updated event state status(E_i)

`// Step 1: Authorization and Retrieval`
Receive verification request for event E_i from verifier a;
Verify $a \in A$ and check authorization via *Authority Manager*;
Retrieve event record ER_i from the *Event Registry*;

`// Step 2: Pre-condition Check`
if status(E_i) $\in$ {CONFIRMED, REJECTED} **then**
  return ABORT: Terminal State Reached;

`// Step 3: Off-chain Integrity Verification`
Retrieve off-chain content using $CID(d_i)$;
Verifier computes $H'(E_i) = \text{SHA-256}(d_i \parallel t_i \parallel s_i)$;
if $H'(E_i) \neq H(E_i)$ **then**
  $V(E_i, a) \leftarrow 0$; `// Identify manipulation`
  Flag event as *Potentially Manipulated*;

`// Step 4: Decision Recording`
Store $V(E_i, a)$ in the *votes* mapping;
Generate signature: $\pi_i = \text{Sign}_{sk_a}(H(E_i), V(E_i, a), t_{verify})$;

`// Step 5: Threshold Evaluation and State Transition`
if $V(E_i, a) = 1$ **then**
  $confirmationCount \leftarrow confirmationCount + 1$;
if $\sum_{a \in A} V(E_i, a) \geq k$ **then**
  status(E_i) $\leftarrow$ CONFIRMED;
else if *Rejection threshold met* **then**
  status(E_i) $\leftarrow$ REJECTED;
else
  status(E_i) $\leftarrow$ UNDER_REVIEW;

`// Step 6: Finalization`
Update *Provenance Tracker* with state transition;
Emit EventVerified($eventId$, status, a, t_{curr});
return status(E_i);

Algorithm 1 and 2 define event registration and verification, including integrity checks, decision submission, and threshold-based state transitions, formalising event preparation, verification, and transition with deterministic and auditable behaviour.

4 Algorithm Design and Formal Event Lifecycle

This section defines the event lifecycle in ChainProv-City, with reporting, verification, action and closure, and ensures controlled, auditable and irreversible transitions. Unlike open or anonymous consensus systems, it adopts an authority-based model aligned with institutional practices, ensuring accountability and provenance.

4.1 Event Reporting

Algorithm 3: Authority-Based Event Reporting

Require: Event data E, authorized gateway identity G_j, whitelist contract W
Ensure : Immutable event record on blockchain with status REPORTED

`// Procedure: REPORTEVENT`(E, G_j)
if isAuthorized(G_j, W) = false **then**
  return ERROR: Unauthorized reporter;

`// Step 1: Integrity and Storage`
$h \leftarrow \text{SHA-256}(E.\text{data} \parallel E.\text{timestamp} \parallel E.\text{source})$;
$cid \leftarrow$ storeOffChain(E.data) ; `// IPFS or encrypted storage`

`// Step 2: Transaction Preparation`
$\tau \leftarrow (h, cid, E.\text{metadata}, G_j.\text{address})$;
$tx \leftarrow$ submitTransaction(reportEvent, τ);

`// Step 3: Consensus and Finalization`
waitForConfirmation(tx) ; `// Blockchain consensus`
$eid \leftarrow$ parseEventID(tx.receipt);
Emit (EventReported, eid, h, E.metadata);

return eid;

Event reporting describes how surveillance events are introduced into the blockchain system to provide an immutable provenance record by linking a hash of off-chain data with metadata. Only trusted reporters (e.g., edge gateways or devices) can report events (via a whitelist in the Authority Manager contract). When submitted, events are created on-chain with status REPORTED and triggering verification. Events are identified by a unique ID (counter and timestamp) for later use. Reporting is detailed in Algorithm

3, including authorisation, hashing, off-chain storage, transaction submission and consensus finalisation. Event notifications enable real-time updates in verifier dashboards.

4.2 Multi-Agency Verification with k-of-m Threshold

Once reported, events undergo multi-agency verification with signed decisions from authorised authorities. Acceptance requires at least *k* approvals from *m* authorities, preventing unilateral validation and enforcing distributed trust. Algorithm 4 formalises this process, coordinating Layer 2 state management and Layer 3 interactions, including off-chain retrieval, validation, voting, threshold evaluation, and timeout handling.

Algorithm 4: Multi-Agency Verification with k-of-m Threshold

Require: Event hash h, set of agencies $\mathcal{A} = \{A_1, A_2, \ldots, A_m\}$, threshold k, timeout Δt
Ensure : Final event decision recorded on blockchain with cryptographic proof

```
// Initialization
approvals ← 0, rejections ← 0, votes ← ∅;
T_start ← currentTime();
// Main Consensus Loop
while currentTime() − T_start < Δt do
    foreach agency A_i ∈ A do
        if hasVoted(A_i, h) = false then
            d_i ← retrieveOffChain(h.cid) ;        // Layer 3 retrieval
            v_i ← A_i.Review(d_i) ;        // Human or automated review
            σ_i ← Sign_{A_i.sk}(h ‖ v_i ‖ timestamp);
            submitVote(h, v_i, σ_i) ;              // Layer 2 transaction
            votes ← votes ∪ {(A_i, v_i, σ_i)};
            if v_i = APPROVE then
                approvals ← approvals + 1;
            else if v_i = REJECT then
                rejections ← rejections + 1;
    // Check Termination Conditions
    if approvals ≥ k then
        finalizeState(h, VERIFIED);
        return SUCCESS: Event verified;
    else if rejections > (m − k) then
        finalizeState(h, REJECTED);
        return FAILURE: Event rejected;
// Timeout Handling
finalizeState(h, DISPUTED);
return TIMEOUT: Escalation required;
```

4.3 Action and Event Closure

Once verified, events may be actioned and move through the lifecycle. When an event is VERIFIED, it may be automatically or by an authorised operator, signalling an action in the physical world (e.g., an emergency call-out). Events finally reach a final outcome recorded as RESOLVED (valid incident) or FALSE_ALARM (false incident or misinterpreted). Transitions are tracked on-chain and the contract logic prevents invalid transitions, providing a stable and resilient event lifecycle.

4.4 Formal State Machine and Visualisation

Figure 1 summarises all possible states and transitions of events.

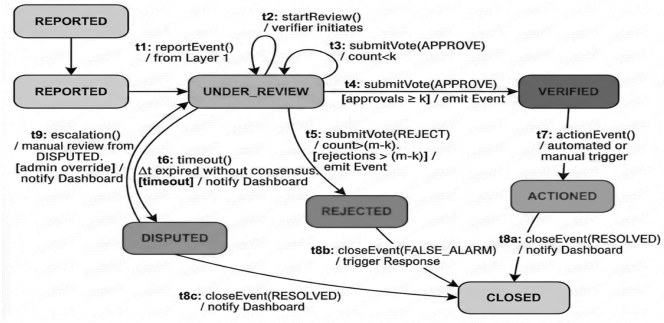


Fig. 1. Event lifecycle showing reporting, multi-agency verification, and final state transitions.

Events start in the REPORTED state and move depending on verification to VERIFIED (threshold reached) or REJECTED/DISPUTED (not enough or conflicting votes). They then progress to ACTIONED and then CLOSED (with a result recorded). This ensures a structured, traceable lifecycle with all states and transitions captured on-chain.

4.5 Access Control and State Transition Rules

Access control and state transition rules ensure only permitted actors are involved and events follow the lifecycle. RBAC, implemented in smart contracts, allocates role-based permissions: gateways report events, agencies vote and authorised users initiate actions or closure. Roles are configured at deployment and securely updated (see Table 2).

Table 2. Role-Based Access Control Matrix.

Role	reportEvent	submitVote	actionEvent	closeEvent	adminUpdate
Edge Gateway	Yes	No	No	No	No
Police Agency	No	Yes	Yes	Yes	No
Fire Agency	No	Yes	Yes	Yes	No
Medical Agency	No	Yes	Yes	Yes	No
Transport Agency	No	Yes	No	No	No
System Admin	No	No	No	No	Yes

4.6 Relationship to System Architecture

Algorithms 3 and 4 reflect the architecture in Section 3. Algorithm 3 is responsible for event reporting, hashing and off-chain storage (Layer 1), and Algorithm 4 for verification and decision making (Layers 2-3). Figure 1 maps to lifecycle changes, and Table 2 maps to the RBAC model.

5 Security and Threat Model

This section defines the security model of ChainProv-City by examining the adversaries, threats and vulnerabilities in smart city surveillance systems. It shows that the approach ensures integrity, accountability and auditability in a practical attack scenario. We adopt a typical permissioned blockchain model, with the majority of the validator nodes being honest, as in a Byzan-tine fault-tolerant system for consortium and government blockchain deployments. We assume that a full validator compromise is out of scope as it is a failure of the institution.

5.1 Adversary Model

The adversary model acknowledges the open and diverse nature of smart city environments, including both external and internal adversaries. It is compatible with models like the CIA triad and STRIDE, covering integrity, availability, and access control.

We classify the adversaries into five groups: external adversaries creating fabricated events (deepfakes, spoofed data, etc.), insiders abusing their privileges, compromised devices with stolen credentials, collusive authorities manipulating the outcomes, and software-based adversaries exploiting smart contract bugs. Adversaries are assumed to have access to system design, but not cryptographic primitives (Kerckhoffs' principle). They cannot forge signatures, reverse secure hashes, or decrypt protected data.

5.2 Threat Analysis and Mitigation

Table 3 summarises the relationship between identified threats, their objectives, corresponding defence mechanisms, and residual risks within ChainProv-City framework.

Table 3. Threat Model Analysis: Adversaries, Goals, and Defences.

Threat and Adversary	Goal and Attack Vector	Defence Mechanism	Residual Risk
Malicious Data Injection (Deepfakes, Sensor Spoofing)	Inject fabricated data to trigger false alerts or disrupt operations.	Authority-based k-of-m multi-agency verification prevents unilateral validation.	Collusion of greater than or equal to k authorities could bypass verification.
Evidence Tampering (Insider, Hacker)	Modify or delete evidence to evade accountability.	Immutable on-chain hashes expose off-chain modification.	Off-chain data deletion is detectable but irreversible.
Device Impersonation	Submit fraudulent events using stolen credentials.	PKI-backed identities and access control at submission time.	Physical key compromise without timely revocation.
Verifier Collusion	Approve false or suppress legitimate events.	Threshold governance and identity-logged approvals ensure accountability.	Majority authority conspiracy representing a non-technical risk.
Denial-of-Service (DoS)	Flood the system to delay or block processing.	Event batching, rate-limiting, and permissioned access.	Extreme-scale flooding may still degrade availability.
Privacy Breach	Unauthorized access to sensitive surveillance data.	Off-chain encryption with only on-chain hashes stored.	Metadata inference and key leakage risks.
Smart Contract Exploits	Exploit logic flaws to alter system behaviour.	Minimal state-machine design limits attack surface.	Undiscovered zero-day vulnerabilities.

The system provides integrity and authorisation via multi-agency verification and immutable on-chain anchoring. The k-of-m threshold prevents data integrity attacks (e.g., malicious data injection) and verifier attacks (e.g., collusion) by requiring a quorum, and cryptographic hashing detects tampering with off-chain data. Identity attacks (e.g., device impersonation) are prevented using PKI-based authentication, while availability risks (e.g., denial-of-service) are mitigated through rate limiting, batching and permissioning. Privacy is enhanced by separation of on-chain and off-chain data, but some risks (e.g., metadata inference) remain. These ensure four things: integrity

(tampering is detectable), authorisation (only permitted parties act), non-repudiation (decisions are signed and recorded), and auditability (the event lifecycle is recorded on-chain). These hold if less than the threshold number of authorities are corrupt. The approach does not prevent non-technical challenges (e.g., institutional corruption) or guarantee off-chain data availability, reflecting trade-offs in governance-based design.

6 Implementation and Prototype

Here we describe the implementation and validation of ChainProv-City. A prototype on Ethereum shows that authority-based multi-agency verification is enforced at the smart contract level without a central authority. The implementation focuses on correctness, governance, and auditability, rather than scalability. Access control, verification and irreversible state transitions are tested, proving the concept.

6.1 Ethical Considerations

Privacy, consent, and security versus rights are issues in blockchain-based surveillance [13]. ChainProv-City is privacy by design, with only hashes stored on the blockchain and raw data stored encrypted off-chain to support auditability and GDPR compliance [14], [15]. The verification operations are tied to institutions, providing accountability and auditability, enabling audit and retrospective scrutiny of bias or abuse [15]-[17]. In public surveillance, consent may not be feasible, so limited disclosure and access ensures minimal exposure while maintaining verifiability. The framework does not cover non-technical risks, like institutional misuse or regulatory failure, as it relies on law.

6.2 Prototype Architecture

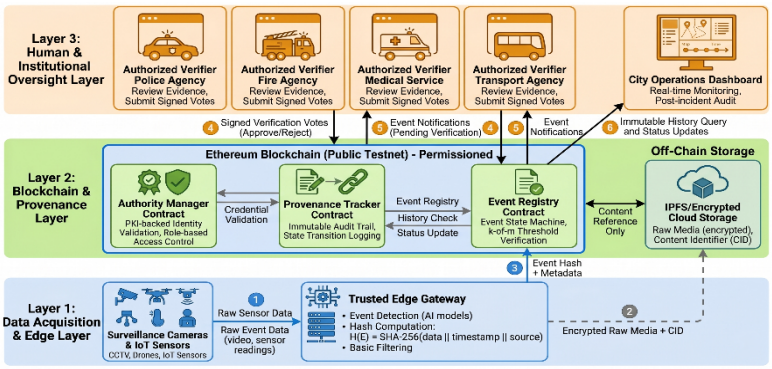


Fig. 2. Three-layer ChainProv-City architecture with edge, blockchain, and oversight layers.

The proposed prototype (Figure 2) uses an on-chain/off-chain architecture to achieve scalability, privacy and auditability. On the data collection layer, IoT and surveillance data is captured and processed by edge gateways, which create hashes and store encrypted evidence off-chain (e.g. IPFS) to create content identifiers (CIDs). The Event Registry, which kick-starts the lifecycle, receives hashes and metadata, and the Authority Manager enforces access control. The Provenance Tracker logs decisions, keeping

a record of state changes and verifier addresses. This approach maintains off-chain data while ensuring verifiability, with dashboards enabling real-time monitoring and audit.

6.3 Validation and Test Results

The framework was validated via automated Hardhat unit tests, confirming enforcement of access control, threshold verification, and valid state transitions.

Table 4. Smart Contract Verification Test Cases and Validation Targets.

Test Case ID	Scenario Description	Expected Behaviour	Validation Target
TC-001	Unauthorised account attempts to verify event	Transaction reverted, no state change	Authorization safety
TC-002	Authorised authority submits valid approval vote	Vote recorded, event state updated	Correct verification workflow
TC-003	Duplicate approval from same authority	Second vote rejected, no double counting	Prevention of double voting
TC-004	Event transitions with k approvals (threshold met)	State changes to VERIFIED	Threshold correctness
TC-005	Event transitions with insufficient approvals	State remains REPORTED	Threshold enforcement
TC-006	Invalid state transition (REJECTED→VERIFIED)	Transaction reverted	Irreversible state transitions
TC-007	Authorised authority verifies after terminal state	Transaction reverted	Terminal state integrity

Test Execution. Tests were executed on a local Ethereum test network with five validator nodes. All seven test cases passed successfully.

```
Figure 3(a) — Invocation & environment
$ cd /Users/maruffarhan/Desktop/Chainprov-city
$ npm test

> chainprov-city@1.0.0 test
> npx hardhat test

WARNING: You are currently using Node.js v25.9.0, which is not supported by Hardhat. This can lead to unexpected behavior. See

Environment (this run): Node.js v25.9.0 | npm 11.12.1 | Hardhat 2.28.6
Solidity 0.8.19 | Hardhat network | chainId 31337
```

Fig. 3a. Test environment and execution setup.

```
Figure 3(b) — Suite & tests TC-001–TC-004
  ChainProv-City: Authority-Based Provenance Framework

    ✔ TC-001: should revert when unauthorised account attempts verification
    ✔ TC-002: should record vote when authorised authority approves
    ✔ TC-003: should reject duplicate votes from same authority
    ✔ TC-004: should transition to VERIFIED when threshold k is met
```

Fig. 3b. Test cases TC-001 to TC-004 results.

```
Figure 3(c) — TC-005–TC-007 & aggregate result
  ChainProv-City: Authority-Based Provenance Framework

    ✔ TC-005: should remain REPORTED with insufficient approvals
    ✔ TC-006: should revert invalid state transitions
    ✔ TC-007: should revert verification attempts after terminal state

  7 passing (2s)
```

Fig. 3c. Test cases TC-005 toTC-007 results.

Figures 3(a)-3(c) demonstrate the successful execution of all test cases, establishing access control, no duplicate voting, threshold enforcement and terminal state integrity.

6.4 Formal Validation of Correctness and Governance

The framework can also be analysed formally to show correctness under the assumed trust model. Let $E=\{e_1, e_2, ..., e_n\}$ be the set of events and $A=\{a_1, a_2, ..., a_m\}$ be the set of authorised authorities. Each event e_i has a state: $\sigma_i \in$ {REPORTED, VERIFIED, DISPUTED, ACTIONED, CLOSED}

Threshold Correctness. Let $V(e_i) \subseteq A$ be the set of authorities that approve event e_i. An event is verified if:

$$|V(e_i)| \geq k \tag{12}$$

This ensures no single authority can approve an event unilaterally. Our tests show events stay in REPORTED until threshold is met.

Authentication and Access Control. A verifier a_j may only act if:

$$a_j \in A \tag{13}$$

Unauthorised access is denied and does not impact state. This guarantees that only authorised institutional entities affect event outcomes.

Irreversible State Transitions. Event states follow a monotonic progression:

$$\sigma_i (t+1) \geq \sigma_i (t) \tag{14}$$

Once an event is in a terminal state, it stays there. This precludes replay attacks, duplicate verification and post-hoc manipulation.

Correctness Guarantees. Under the assumption the number of malicious authorities is less than *k*, no false events can be verified without collusion, decisions are traceable to identifiable entities, and the provenance record is immutable and auditable.

7 Case Study: Deepfake Riot Alert in a Smart City

This section offers a case study demonstrating how ChainProv-City reacts to an adversarial scenario involving AI-generated disinformation. The focus is not on accuracy, but rather on governance, multi-authority verification and accountability, as per the validation process in Section 6.

7.1 Attack Scenario

An adversary injects a deepfake event through a compromised edge gateway with stolen credentials. The event is hashed, stored off-chain and submitted as a REPORTED event. In a centralised approach, this may lead to false alarms. In ChainProv-City, this is avoided by multi-authority verification. Each authority examines the event, identifies contradictions and casts their vote to reject. Since the approvals remain below the

threshold, the event is not VERIFIED and no automatic action is taken. The decision-making process is recorded on the blockchain, making it transparent.

7.2 7.2 Multi-Authority Detection and Response

The event results in notifications to verification authorities. Each verification agency separately fetches the off-chain media and checks the hash. Verification involves automated verification (e.g., deepfake detection, multiple camera comparison) and contextual verification. Conflicts are identified, resulting in signed rejections being submitted on-chain. The event is escalated for human intervention, and more inconsistencies are detected. Since approvals remain below the threshold, the event does not move to VERIFIED but stays in REPORTED. Thus, no automatic actions are taken. All decisions (including explanations) are logged, enabling auditability and attribution.

7.3 Mapping to Verification and Test Cases

This case study directly corresponds to the test scenarios (TC-001 to TC-007) defined in Table 5 and validated in Section 6.

Table 5. Case Study Event Mapping to Test Case Validations.

Case Study Event	Test Case	Validation Demonstrated
Adversary injects deepfake via compromised gateway	TC-001	Unauthorised account cannot verify; event enters REPORTED state but cannot advance without legitimate authority votes
Legitimate authorities receive notification	TC-002	Authorised authorities can access and interact with submitted events
Multiple authorities review and reject	TC-003	Duplicate vote prevention ensures one vote per authority; consensus emerges independently
Rejection threshold reached	TC-004 (inverse)	Threshold mechanism prevents VERIFIED state when k approvals are not achieved
Event remains in REPORTED state	TC-005	Event remains in REPORTED state without threshold met; no false escalation occurs
Attempted approval after rejection	TC-006	Invalid state transition prevented; smart contract enforces lifecycle constraints
Final state persistence	TC-007	No further modifications possible; terminal state integrity is maintained

7.4 Robustness and Limitations

The case study illustrates the key properties but some remain. Our framework assumes reliable detection tools but focuses on verification. Verification delays depend on processes and may not be instantaneous, and the model assumes secure key management, where compromised credentials may remain valid until revoked. These constraints suggest further research in areas such as linking with forensic tools, improving verification processes, and more robust credential management. The framework, despite these limitations, exhibits an important feature: no single point of failure. Decentralised verification avoids false alarms even when parts of the system are compromised.

8 Discussion and Comparative Analysis

8.1 Meeting the Research Goals and Gap

As illustrated in Table 1, existing approaches do not collectively meet key criteria for trustworthy AI-based surveillance systems, namely real-time, multi-agency, comprehensive, privacy-preserving, and resilient to adversarial manipulation. The authority-based model, formal validation, threat analysis, scenario evaluation and prototype of ChainProv-City address these, bridging the gap identified in Section 2 by providing a unified approach to governance, verification and provenance.

8.2 Related Work

Existing blockchain provenance systems use permissionless models, where verification is done through anonymous consensus or incentives. These approaches work for permissionless systems, but are not well suited to smart city governance, which requires accountability and legal liability. Existing smart city systems use centralised databases, which while efficient, are prone to single points of failure, lack transparency and are vulnerable to insider attacks. ChainProv-City enforces authority-based verification, limiting participation to registered agencies and tying decisions on an immutable ledger. This ties trust to institutions while enabling tamper-proof auditing.

8.3 Practical Considerations and Limitations

The framework is lightweight, using structured state transitions, RBAC, and immutable logging instead of complex cryptography. This enhances deployability in cost and interoperability constrained deployments. Our prototype provides strong provenance assurances without additional infrastructure. ChainProv-City assumes fewer than the threshold number of authorities are malicious, so mass collusion is a non-technical threat. The approach also prioritises provenance over correctness and storage. Off-chain reliability and privacy are complementary, allowing for realistic security guarantees.

9 Conclusion and Future Work

In this paper, we propose ChainProv-City, a blockchain-based provenance framework for smart cities requiring accountability, auditability and institutional trust. The framework integrates governance into verification, unlike open models, to ensure only trusted parties validate events. Theoretical modelling, algorithm and verification demonstrate correctness and irreversible state changes, and threat modelling shows the framework is robust in realistic scenarios. By combining centralised and open technologies, ChainProv-City offers transparency and accountability while deployable.

9.1 Future Work

Future research will involve adaptive verification policies that dynamically set thresholds, improved privacy techniques such as selective disclosure and off-chain computing, and interoperability to support cross-city and cross-regulatory use.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Rahman, M.A., Rashid, M.M., Hossain, M.S., Loukas, G., Hassanain, E., Alhamid, M.F.: Blockchain and IoT-based cognitive edge framework for sharing economy services in a smart city. IEEE Access 7, 18611–18621 (2019)
2. Esposito, C., Ficco, M., Gupta, B.B.: Blockchain-based authentication and authorization for smart city applications. Inf. Process. Manag. 58(2), 102468 (2021)
3. Xie, J., Tang, H., Huang, T., Yu, F., Chen, X.: A survey of blockchain technology applied to smart cities: Research issues and challenges. IEEE Commun. Surv. Tutor. 21(3), 2794–2830 (2019)
4. Akbarfam, A.J., Heidaripour, M., Maleki, H., Dorai, G., Agrawal, G.: ForensiBlock: A provenance-driven blockchain framework for data forensics and auditability. In: Proc. IEEE TPS-ISA, pp. 136–145 (2023)
5. Rahman, M.A., Saifullah, A.: Transparent and tamper-proof event ordering in Internet of Things platforms. IEEE Internet Things J. 10(6), 5335–5348 (2022)
6. Kshetri, N.: Blockchain and smart city governance: A review and future research agenda. Gov. Inf. Q. 38(4), 101567 (2021)
7. Rashid, M.M., Lee, S.H., Kwon, K.R.: Blockchain technology for combating deepfake and protecting video/image integrity. J. Korea Multimed. Soc. 24(8), 1044–1058 (2021)
8. Sun, Y., Li, H., Liu, Q.: A blockchain-based framework for secure multimodal deepfake detection. IEEE Trans. Inf. Forensics Secur. 18, 2345–2358 (2023)
9. Heidari, N.J., et al.: A novel blockchain-based deepfake detection method using federated and deep learning models. Cogn. Comput. 16, 1073–1091 (2024)
10. Zheng, Z., Xie, S., Dai, H., Chen, X., Wang, H.: Blockchain challenges and opportunities: A survey. Int. J. Web Grid Serv. 14(4), 352–375 (2018)
11. Kulothungan, K., et al.: Using blockchain ledgers to record AI decisions in IoT. Preprints (2025)
12. Al-Turjman, F., et al.: Privacy and security in cognitive cities: A systematic review. Appl. Sci. 11(10), 4471 (2021)
13. Zyskind, G., Nathan, O., Pentland, A.: Decentralizing privacy: Using blockchain to protect personal data. In: Proc. IEEE Security and Privacy Workshops, pp. 180–184 (2015)
14. European Union: General Data Protection Regulation (GDPR) (EU) 2016/679 (2016)
15. Jobin, A., Ienca, M., Vayena, E.: The global landscape of AI ethics guidelines. Nat. Mach. Intell. 1, 389–399 (2019)
16. Mittelstadt, B.D., Allo, P., Taddeo, M., Wachter, S., Floridi, L.: The ethics of algorithms: Mapping the debate. Big Data Soc. 3(2) (2016)
17. O'Neil, C.: Weapons of Math Destruction: How Big Data Increases Inequality and Threatens Democracy. Crown Publishing (2016)